DEVELOPIFY

Mobile Developer Handover

Investor + Landowner
Application 1.0.0+6 · Handover revision r1
22 September 2026

Prepared for Ngoako Masetla
Setup, architecture, API integration and service recommendations

The accepted UI is delivered in runnable demo mode. Production APIs, service configuration and distribution signing remain integration work.

Markdown in the source ZIP is the editable source of truth. Source references in this PDF use paths relative to its developify-mobile/ root.

Contents

Developify Mobile 3

Setup guide 5

Architecture 9

API integration guide 14

Observability and notifications 21

Handover status 25

Source revision: 0f097fa9fc6998260fce3dd5692d1011ac778422

Developify Mobile

One Flutter application for **Investor and Landowner**, with shared authentication and portal switching. Application version **1.0.0+6**, source handover revision **r1**. The accepted UI includes the compact Investor calculator banner at standard and enlarged system text settings. Screens and domain journeys are implemented; repositories simulate data and financial operations. Production API integration, service configuration and distribution signing remain the receiving team's responsibility.

Quick start

Use **Flutter 3.44.4 stable / Dart 3.12.2**. Install the native prerequisites in the setup guide. Extract the source ZIP, then run these commands inside `developify-mobile/`:

```
flutter --version
flutter pub get --enforce-lockfile
dart run build_runner build
flutter devices
flutter run --target lib/main.dart \
  --dart-define=MOCK_MODE=true \
  --dart-define=MOCK_SCENARIO=success \
  --dart-define=FIREBASE_ENABLED=false \
  --dart-define=TELEMETRY_ENABLED=false \
  --dart-define=ONESIGNAL_APP_ID=
```

Select the Android emulator or iOS simulator when prompted, or add `-d DEVICE_ID` using an ID returned by `flutter devices`. Both portals are available in the app switcher; switching does not require another login. An authenticated cold restart starts on Investor Home.

Mock-only sign-in: `landowner.qa@developify.co.za` / `Developify@123`. Recovery/verification code: `123456`. Seeded transaction PIN: `1234`. The mock accepts any valid email and nonempty password; these are convenient test inputs, not production credentials. A changed PIN lasts for the current user in the running process and resets on logout, user change or restart. Recovery does not create a real server password. Login restoration does not persist the simulated wallet, submissions or KYC data; see the persistence matrix.

Guides

- Setup guide: toolchain, configuration, commands and signing.
- Architecture: ownership, repositories, navigation and persistence.
- API integration guide: concrete interfaces, backend contracts and verification.
- Observability and notifications: existing adapters and the recommended next phase.
- Handover status: delivered scope, verification and integration inputs.

The standalone developer PDF is a reading copy of this documentation. Markdown is the editable source of truth. The exported .source-revision records the exact commit; it is expanded by Git archive, not a runtime configuration value.

Stack and structure

Flutter/Dart, MobX with committed generated source, GetIt constructor composition, go_router, Dio, ScreenUtil, flutter_svg and flutter_secure_storage. Firebase and OneSignal adapters are optional and unconfigured in this delivery. Dependency versions are retained in pubspec.lock [source: pubspec.lock]; CocoaPods dependencies are retained in ios/Podfile.lock [source: ios/Podfile.lock].

```
lib/main.dart          Normal application entry point
lib/app/               Bootstrap, DI, environment and router
lib/core/              Money, errors, network, storage, services, theme, controls
lib/features/          Authentication, Investor, Landowner, profile and notifications
assets/                Runtime SVGs, images and licensed TASA fonts
test/                  Unit, widget, journey and architecture regressions
integration_test/      Maintained native device journeys
test_driver/           Native integration driver
tool/                  Launcher and splash resource generators
android/ and ios/       Native applications and build configuration
docs/                  The five handover guides
```

Common commands

```
dart run build_runner build
dart format --output=none --set-exit-if-changed \
  lib test integration_test test_driver tool
flutter analyze
flutter test --reporter expanded
```

MobX `.g.dart` files are tracked; never edit them manually. The pinned build_runner does not need the obsolete `--delete-conflicting-outputs` flag. There is no package upgrade required to run this handover.

For normal Android debug and iOS simulator builds, use the exact disabled-service commands in Build and native tests. Android release-mode builds currently use debug signing for local testing. No production APK, AAB or IPA is part of the source handover.

The app uses local runtime assets and one font family, TASA Orbiter Display. Its OFL licence [source: assets/fonts/TASA-Orbiter-OFL.txt], SVG identifiers, native launcher assets and splash resources are preserved. Regenerate launcher resources only when intentionally updating branding with `flutter test tool/generate_launcher_icons.dart`; verify with `flutter test test/launcher_icons_test.dart`. See the setup guide for optional test diagnostics.

Setup guide

Documentation entry point

Tested toolchain

Component	Source expectation and tested installation
Flutter	3.44.4 stable, framework revision `ad70ec4617`
Dart	3.12.2, supplied by Flutter
Repository constraints	`sdk: ^3.12.2`, `flutter: >=3.44.4`; minimum constraints, not an exact SDK manager pin
Java	Microsoft OpenJDK 17.0.17; Java/Kotlin JVM target 17
Android	compile/target SDK 36, minimum SDK 24; NDK 28.2.13676358 from this Flutter SDK
Gradle / AGP	Wrapper 9.1.0 / Android Gradle Plugin 9.0.1
Kotlin	2.3.20 declared in settings; existing Flutter compatibility flags retained
Apple tooling	Xcode 27.0 (27A266a), CocoaPods 1.16.2
iOS	Deployment target 15.0; CocoaPods integration; Swift Package Manager disabled in pubspec

Use this tested Flutter version first. Do not automatically upgrade Flutter, Gradle, Kotlin or pods during integration. The host also has newer Android SDK platforms; the app resolves API 36 from Flutter. `flutter doctor` reports Android licence status as unknown on the verification host; actual build outcomes are recorded in handover status. On a new workstation, use `flutter doctor -v` and review/accept applicable SDK licences as the responsible developer.

Native identifiers were checked independently: Android `za.co.developify.developify_mobile` in app/build.gradle.kts [source: android/app/build.gradle.kts], and iOS `za.co.developify.developifyMobile` in project.pbxproj [source: ios/Runner.xcodeproj/project.pbxproj]. They are provisional distribution identifiers; confirm them before registering production apps. The two portals remain one app per environment.

Clean extraction and dependencies

Extract the ZIP into an ordinary writable directory. Enter its single `developify-mobile/` root; no files from another checkout are required. Preserve dotfiles and executable permissions. Retained Gradle wrapper scripts/JAR and native projects are included; Flutter regenerates machine-specific files locally.

```
flutter --version
flutter pub get --enforce-lockfile
dart run build_runner build
dart format --output=none --set-exit-if-changed \
  lib test integration_test test_driver tool
flutter analyze
flutter test --reporter expanded
```

`--enforce-lockfile` is supported by the tested Flutter/Pub toolchain. It must fail if locked resolution cannot be honoured; do not delete or regenerate the lockfile to hide a mismatch. Pub and native dependency downloads require network access on a machine without caches. Run commands from the extracted root

so asset and test paths resolve correctly.

For iOS on macOS, run `flutter pub get --enforce-lockfile` before `(cd ios && pod install --deployment)`. The deployment flag enforces Podfile.lock [source: ios/Podfile.lock]. Open `ios/Runner.xcworkspace`, not the `.xcodeproj`, for native development. Select a locally installed compatible simulator using `flutter devices` or Xcode. A physical iPhone additionally requires an approved signing team and provisioning.

Configuration

AppEnvironment.fromDefines [source: lib/app/environment/app_environment.dart] reads compile-time `--dart-define` values. There is no `.env` loader. Values are embedded in the application and must never contain server/admin secrets.

Key	Parser, default and meaning	Safe example
`MOCK_MODE`	`bool.fromEnvironment`, defaults true. False makes registered mock operations unavailable; does not install live repositories.	`true`
`MOCK_SCENARIO`	String mapped to the cases below; unknown values fall back to success.	`success`
`MOCK_DELAY_MS`	`int.fromEnvironment`, defaults 350 milliseconds; malformed values use the default. Use a nonnegative value; the parser itself does not clamp negatives.	`350`
`API_BASE_URL`	String, defaults empty. `apiUri` accepts only HTTPS with host and no user-info, query or fragment. Only needed once API adapters are implemented.	`https://api.example.com` (illustration only)
`FIREBASE_ENABLED`	`bool.fromEnvironment`, defaults false. Attempts optional Firebase initialization when true; configuration failure falls back to no-ops.	`false`
`TELEMETRY_ENABLED`	`bool.fromEnvironment`, defaults false. Gates analytics/crash/performance collection after Firebase initialization.	`false`
`ONESIGNAL_APP_ID`	String, defaults empty. Empty/whitespace disables initialization; a nonempty value attempts initialization independently of mock/Firebase flags.	Explicit empty value

Supported scenario strings: `success`, `empty`, `network_failure`, `low_balance`, `incomplete_kyc`, `pending_verification`, `rejected_verification`, `no_projects`, `no_transactions`, `no_notifications`, `no_portfolio`, `withdrawal_unavailable`, `payment_method_unavailable`, `validation_failure`, `missing_item`. Applicability varies by repository. `network_failure` fails the first request on each affected repository behavior and then permits retry.

For bool defines, use literal `true` or `false`. Keep the explicit empty OneSignal define in verification commands, and avoid personal IDE launch profiles or `--dart-define-from-file` inputs. No approved Firebase client options, Google services files or OneSignal App ID are supplied. An API URL by itself does not connect auth or any repository.

Build and native tests

Normal mock launch:

```
flutter run --target lib/main.dart \
  --dart-define=MOCK_MODE=true \
  --dart-define=MOCK_SCENARIO=success \
  --dart-define=FIREBASE_ENABLED=false \
  --dart-define=TELEMETRY_ENABLED=false \
  --dart-define=ONESIGNAL_APP_ID=
```

Normal Android build (temporary validation output):

```
flutter build apk --debug --no-pub --target lib/main.dart \
  --dart-define=MOCK_MODE=true \
  --dart-define=MOCK_SCENARIO=success \
  --dart-define=FIREBASE_ENABLED=false \
  --dart-define=TELEMETRY_ENABLED=false \
  --dart-define=ONESIGNAL_APP_ID=
```

iOS simulator build command, without distribution signing (currently blocked on the verification host; see the Xcode 27 limitation below):

```
flutter build ios --simulator --debug --no-pub \
  --target lib/main.dart \
  --dart-define=MOCK_MODE=true \
  --dart-define=MOCK_SCENARIO=success \
  --dart-define=FIREBASE_ENABLED=false \
  --dart-define=TELEMETRY_ENABLED=false \
  --dart-define=ONESIGNAL_APP_ID=
```

Native journeys use independent test composition and deterministic fixture data:

```
flutter drive --no-pub \
  --driver=test_driver/integration_test.dart \
  --target=integration_test/client_review_test.dart -d DEVICE_ID
flutter drive --no-pub \
  --driver=test_driver/integration_test.dart \
  --target=integration_test/landowner_flow_test.dart -d DEVICE_ID
```

Replace `DEVICE_ID` with the actual ID. The first journey covers Investor money/privacy/project flows and portal switching; the second covers Landowner drafts and submission. Also retain `calculator_banner_test.dart`, `investor_flow_test.dart`, `investor_auth_flow_test.dart`, `investor_verification_test.dart`, `investor_amount_test.dart` and `investor_keyboard_test.dart` for focused native checks. Read their setup before running, since they deliberately create test sessions rather than verify a normal production login.

Optional diagnostics are build/test tooling, not application configuration: `CAPTURE_FIDELITY=true` enables screenshot captures for widget regressions; `FIDELITY_OUTPUT` overrides their default `build/test-artifacts/screenshots` location. `CAPTURE_SCREENSHOTS=true` enables basic app-test captures. `CAPTURE_NATIVE_SCREENSHOTS=true` enables optional native captures where supported. The integration driver writes `build/integration-results`, overridable with the shell variable `DEVELOPIFY_TEST_OUTPUT`. `LAUNCHER_PREVIEW_DIR` optionally writes icon previews when running the launcher generator. Generated diagnostics are not required input fixtures.

Troubleshooting

- Generation: rerun `dart run build_runner build`; never hand-edit `.g.dart`. A stale generated file is a source consistency problem, not a reason to disable analysis.
- Assets: run from the root after Pub resolution. Check AppAssets [source: lib/core/assets/app_assets.dart], pubspec [source: pubspec.yaml] and exact filename case. Do not substitute missing runtime art with removed design references.
- Android: verify Java 17, installed SDK 36/NDK, network and wrapper permissions. `android/local.properties` is created for the current machine. Existing AGP compatibility flags in gradle.properties [source: android/gradle.properties] are intentional; this handover does not migrate the build

system.

- iOS: resolve Pub first, then `pod install --deployment`. A lock mismatch needs investigation, not `pod update`. Ensure the selected simulator runtime is installed. Keep the workspace's deployment target at least 15.0 for the pinned Firebase dependencies.
- Optional services: disabled services are expected. With Firebase enabled, initialization errors yield no-ops. Provide approved platform options and native setup before expecting dashboards. Consult the service guide.
- Native signing: simulator success does not prove device or store signing. Android `release` currently uses the local debug identity; a new developer's debug identity can differ. Do not distribute that build as production or overwrite installed apps with incompatible signing.

Current Xcode 27 limitation

The normal universal simulator build failed in Flutter framework architecture verification (`debug_unpack_ios`, Xcode exit 255). A selected iPhone 17 / iOS 26.5 simulator attempt also failed under Xcode 27 with generated dependency targets below the supported iOS 15.0 minimum. No successful fresh iOS build or native journey is claimed for this handover host. The app target itself remains iOS 15.0. Do not silently migrate the native project or upgrade dependencies to hide this result; the receiving team should verify with an agreed compatible Xcode/toolchain before iOS distribution. Android builds and host tests are independently verified.

Production distribution and environments

The CTO owns package/bundle confirmation, signing certificates/keystores, store accounts, privacy declarations, service registrations and CI access. No private signing material is included. Plan approved development, staging and production configuration with independent service projects and clear release labels. Keep credentials in controlled CI secret storage, not `--dart-define` or source files. See the recommended environment workflow.

Release-mode QA is useful for performance validation with mock data; it is separate from production signing, server integration and store deployment. Source delivery requires none of those external activations.

Architecture

Documentation entry point

Ownership and request flow

```
main -> bootstrap -> AppDependencies (one private GetIt container)
                  -> SessionService -> router authentication guards
                  -> optional telemetry / push / ApiClient
                  -> repository singletons and independent store factories

route/widget owns controllers and its store
    -> store validates UI input and manages observable load state
    -> repository interface returns Result<domain model>
    -> mock data today; authenticated API adapter in the next phase
    -> immutable models / change stream
    -> store ignores stale results and updates the view
```

bootstrap [source: lib/app/bootstrap.dart] restores the mock session before composition and installs framework/platform error handlers only when launching. AppDependencies.create [source: lib/app/di/app_dependencies.dart] owns registrations and disposal. It uses `GetIt.asNewInstance`, singleton repositories/services and factories for route-owned stores. Widgets own/dispose text, scroll, animation and focus controllers. Stores own/dispose subscriptions, reactions, timers and request generations. `AppDependencies.dispose` resets the container; `DevelopifyApp` owns its router unless one was injected.

Stores never import, inject, observe or call another store. They receive repository/service interfaces through constructors. Shared repositories and immutable results coordinate features. The architecture tests [source: test/architecture_test.dart] enforce store independence, primitive routing, local assets, centralized colours and typography.

Feature map

Source folder	Responsibility
app [source: lib/app]	Environment, initialization, constructor composition, named routes and shells
core [source: lib/core]	`Result`, cancellation/lifecycle, `MonetaryAmount`, HTTP, storage, telemetry/push, shared theme and controls
authentication [source: lib/features/authentication]	Shared sign-in/recovery and portal registration models/stores
investor [source: lib/features/investor]	Catalogue, project content, accounts, transfers, holdings, independent feature stores and four KYC flows
landowner [source: lib/features/landowner]	Property states, submission drafts, earnings, independent Home/properties/submission stores
notifications [source: lib/features/notifications]	Landowner notification list and local chat
profile [source: lib/features/profile]	Shared profile, preferences, payout account and simulated security controls
shared [source: lib/features/shared]	`AppUser`, `EntityType`, `Experience` and experience selection
onboarding [source: lib/features/onboarding]	Shared onboarding state and presentation

Investor KYC subfolders contain its plan, typed field/member/document models and validation. Core money and errors are reused; domain behavior is not placed inside a global application store. API integration maps every repository and method group.

Routing and shared identity

There is one `MaterialApp.router`, initialized with a 393 x 852 ScreenUtil design reference. createAppRouter [source: lib/app/router/app_router.dart] owns public/protected redirects, pending protected locations, the Landowner shell and Investor routes. investor_routes.dart [source: lib/app/router/investor_routes.dart] composes the Investor tab shell and feature routes. Stable route names live in route_names.dart [source: lib/app/router/route_names.dart]. Routes pass primitive IDs and scalar query values; domain objects are resolved through repositories rather than passed in nonserializable `extra`.

`SessionService` is a non-MobX shared authentication service. `Experience` selects Investor or Landowner within that login. `InvestorAccountContext` adds entity scope (`individual`, `company`, `trust`, `stokvel`) and a revision that invalidates results when scope changes. An entity selection updates the saved user's entity; entity-specific working data remains in memory. The server must independently authorize membership, roles and resources for every request. Client selection is not authorization.

The router remembers a protected location encountered before login and clears it for explicit logout. This supports existing in-process redirects, but is not a complete cold-start notification queue. Push initialization/readiness and server resource authorization still need integration.

Errors, cancellation and financial ownership

Result [source: lib/core/errors/result.dart] is `Success<T>` or `Failure<T>(AppFailure)`. Failures carry a typed kind, user-facing message, optional HTTP code and field errors. Views use `LoadStatus.initial/loading/success/empty/error`; cancellation should not be presented as a failed user action.

Lifecycle [source: lib/core/utils/lifecycle.dart] advances a generation on every new request and on disposal. A completed old request cannot overwrite newer state. Cancellation [source: lib/core/utils/cancellation.dart] and MockRequestScope [source: lib/core/services/mock_request_scope.dart] cancel outstanding operations; Investor repositories additionally check `InvestorAccountContext.isCurrent`. Keep these ownership rules when adding Dio `CancelToken` handling. See the concurrent favourite/detail, account-scope and logout regressions before modifying repository streams.

`MonetaryAmount.minorUnits` stores exact cents with an explicit currency. Multiplication/rounding uses integer/BigInt operations. Current investment policy is ZAR, minimum R1 and a 36-month monthly simple-interest preview. `InvestmentProjection` also represents historical cadences; its constructor default is quarterly, so a new adapter must explicitly pass the actual schedule. Round total earnings once and allocate residual cents across payments. For R100 at 14.5% over 36 months, total earnings are R43.50: thirty R1.21 and six R1.20 payments. Principal returned at maturity is not another credit of aggregate proceeds.

InvestorAccountSummary.from [source: lib/features/investor/domain/account_summary.dart] computes Home overall balance from available ZAR wallet funds plus distinct holdings that still hold principal. It excludes projected earnings and avoids counting a transfer or duplicate holding twice. Historical schedules and settled amounts come from recorded models; a projection is not a ledger entry.

`InvestmentStore` preserves project ID, entered amount, agreement and quote through funding/review; repeated authorization is guarded. The mock account repository rechecks PIN, KYC, current project terms, capacity and balance and supports operation replay signatures. These are domain regressions to preserve, not production security guarantees. A live backend must implement atomic capacity reservation, transaction authorization and idempotent financial writes.

Persistence and session boundaries

State / owner	Portal or entity switching	Process restart	Explicit logout / expiry
Mock login, onboarding, returning user / `SessionService` + `SecureSessionStorage`	One login; selected experience does not replace identity	Secure JSON restores unexpired user, onboarding and selected user entity; experience starts Investor	Sign-out clears active user/expiry, retains onboarding and returning-user record; default session duration is 24 hours
Access token / `TokenStorage`	Shared abstraction	Mock uses `MemoryTokenStorage`; live mode selects secure token storage, but auth/refresh adapters are absent	`AppDependencies.logout` calls `clearToken`; expiry alone calls session sign-out, not the full async logout coordinator
Investor entity / `InvestorAccountContext`	Per-entity data retained; revision invalidates old requests	Selected user entity restored; per-entity residency history is rebuilt	User change resets scope and residency cache
Wallet, transactions, holdings, payment methods / mock account repository	Per entity; preserved within running user session	Fixtures re-created; simulated financial changes are not durable	User change/sign-out clears account maps via context
Investor investment/transfer drafts / `InvestorDrafts`	Keyed by entity + project or transfer kind	Lost	Cleared on user change/sign-out; completed flow can clear its own draft
Favourites and searches / mock catalogue	Per entity, preserved across portal switches	Defaults/fixtures restored	Cleared on user change/sign-out; catalogue fixture contributions are process-local, not a durable backend ledger
Investor KYC / mock KYC repository	Entity drafts remain in current process	Fixtures/default state re-created	User change/sign-out invalidates requests and clears scoped draft data
Investor privacy / mock account repository	Shared across Investor views for current entity	Defaults restored	Reset with the account map
Landowner properties, submissions, drafts and privacy / mock Landowner repository	Shared within process; portal switch preserves them	Fixtures restored; saved local drafts/submissions lost	Explicit `AppDependencies.logout` calls `clearSession`. Expiry alone does not call this repository's cleanup
Profile preferences, security toggles, payout account / mock profile repository	Shared within process	Defaults restored; user profile fields in session JSON can survive	Explicit logout resets repository state. Expiry alone does not invoke its cleanup
Landowner notifications/chat / mock notification repository	Shared within process	Seeded content restored	Explicit logout clears local changes; expiry alone does not invoke repository cleanup

State / owner	Portal or entity switching	Process restart	Explicit logout / expiry
PIN / `MockTransactionAuthorization`	Current authenticated user; portal/entity switching does not reset it	Seed 1234 restored	Resets to 1234 on user change/sign-out; no real biometric enrollment

Expiry monitoring starts in `DevelopifyApp`. Its timer signs out the session; Investor context listeners clear scoped state. Full logout additionally clears Landowner/auth/profile/notification repositories, token storage and push identity. Production integration must unify expiry/revocation with the agreed cleanup coordinator, including pending writes, push unbinding and telemetry identity cleanup. This is documented integration work; the accepted behavior was preserved.

Presentation invariants

The accepted calculator promotion keeps a 370:92 composition scaled once from available width. Only its two live `Text` widgets use `TextScaler.noScaling`; other app text retains system accessibility scaling. Its SVG crop, rounded clipping, reachable tap target and screen-reader text remain. The inactive portfolio promotion is retained as an intentional alternate state.

Privacy uses literal `****` and hidden-value semantics; masking must remove amounts from accessible labels as well as paint. Keep the client policy [source: test/client_review_policy_test.dart], privacy/security state [source: test/security_state_test.dart], responsive [source: test/investor_responsive_test.dart] and banner layout [source: test/calculator_promotion_layout_test.dart] checks when integrating data.

Runtime SVGs, raster crops, meshes, native icons, splash and licensed fonts are retained. Optional screenshot output is diagnostic only. Maintained tests load normal source/native assets, without depending on private review files.

API integration guide

Documentation entry point

For Ngoako Masetla and the receiving engineering team. This is a map of the delivered interfaces and integration responsibilities, **not a working OpenAPI specification**. No production endpoints, provider sessions or deployment pipelines are implemented by this document.

Integration status and composition

All repository registrations are in AppDependencies.create [source: lib/app/di/app_dependencies.dart]. Mock classes under each feature's `data/` folder are registered against interfaces under `domain/`. They preserve loading, empty, validation, retry, cancellation and simulated domain behavior without infrastructure. `MOCK_MODE=false` sets `MockBehavior.available=false`; a configured URL does not substitute live repositories.

Replace implementations at this composition root while keeping independent stores, route ownership and typed models. Add agreed server contracts before financial or KYC writes. Some current interfaces deliberately expose synchronous local draft/simulation operations. They are not ready-made network endpoints: introduce explicit asynchronous persistence/quote contracts where required, with targeted tests, rather than hiding unawaited network writes behind synchronous setters.

Inputs the CTO must supply

- Development/staging/production base URLs, TLS and environment ownership; agreed API schema/versioning and example responses.
- Identity provider and auth/recovery/social-login contract; token issuance, refresh rotation, expiry, revocation and logout rules. Firebase Authentication is not assumed.
- User/entity membership, roles, object authorization, account selection and session policy for both portals.
- Exact money/rate/fee, quote expiry, capacity, idempotency, ledger, settlement, reconciliation and error specifications.
- Pagination/sorting/filtering, resource IDs, status transitions, timestamps/time zones, server validation field keys and rate limits.
- KYC provider/environment, four entity workflows, document storage, upload limits, review/signature/retention rules and callback verification.
- Payment provider sandbox, tokenization/authorization flow, bank verification and authenticated webhook access.
- Push provider and installation-registry contracts, service project/app identifiers, monitoring owners, privacy/consent rules and signing/CI access.

The Flutter app calls authenticated APIs. It must never connect directly to the production database or carry server credentials.

Integration map

Every area below uses the shared composition root above. Method signatures and models linked here are authoritative; pagination and transport schemas are backend decisions, not currently exposed as cursors in the delivered list interfaces.

Authentication and recovery

Contract: AuthRepository [source: lib/features/authentication/domain/auth_repository.dart]. Registered implementation: MockAuthRepository [source: lib/features/authentication/data/mock_auth_repository.dart].

`signIn(email, password)` and `socialSignIn(provider)` return `Result<AppUser>`. `registerLandowner(LandownerRegistration)` also returns a user. Investor signup is `beginInvestorRegistration(InvestorRegistration) -> Result<String>` challenge ID, `verifyInvestorRegistration(challengeId, code) -> Result<void>`, then `completeInvestorRegistration(challengeId) -> Result<AppUser>`; cancellation accepts `Cancellation`, with `cancelInvestorRegistration` and `clearSession` cleanup. Recovery methods request an email challenge, verify a code and reset a password, each returning `Result<void>`.

Backend work: agreed credentials/challenges, real social provider tokens, challenge expiry/rate limits, validation fields and token issuance. The current registration and recovery calls do not prove an identity or send messages. Keep auth errors safe and distinguish invalid input from expired/forbidden/transport failure. No pagination applies.

Prove replacement with authentication tests [source: test/investor_authentication_test.dart], signup tests [source: test/investor_signup_test.dart], mock login/session tests [source: test/qa_login_test.dart] and native auth journey [source: integration_test/investor_auth_flow_test.dart]. Add staging tests for expired challenge, wrong/used OTP, cancellation, duplicate submission, revoked session and refresh races.

User, accounts and selected entities

Contracts: AppUser / EntityType / Experience [source: lib/features/shared/domain/app_user.dart], SessionService [source: lib/core/services/session_service.dart], InvestorAccountContext [source: lib/features/investor/domain/investor_account_context.dart], and `ProfileRepository.load/update`. `selectEntity(entity, nonResident: ...)` changes local scope; `isCurrent(InvestorScope)` rejects stale results. `InvestorAccountRepository.snapshot()` returns `Result<InvestorAccountSnapshot>`.

Backend work: authorized account/entity catalogue, selected membership, profile and residency data, roles and resource-level authorization. Derive protected ownership from authentication and validate every supplied entity/resource identifier. A portal switch is not a new login. Current selected-entity state is not proof of membership. Introduce pagination only if the membership catalogue needs it, with stable IDs and selection invalidation.

Prove with context tests [source: test/investor_context_test.dart], account repository tests [source: test/investor_account_repository_test.dart] and routing tests [source: test/investor_routing_test.dart]: change entity during load, logout during a write, restore a session and reject inaccessible IDs without leaking another entity's data.

Investor catalogue, favourites and search

Contract: InvestorCatalogRepository [source: lib/features/investor/domain/investor_repository.dart]. Implementation: MockInvestorCatalogRepository [source: lib/features/investor/data/mock_investor_catalog_repository.dart]. `projects() -> Result<List<InvestorProject>>`, `project(id) -> Result<InvestorProject>`, `currentProject(id) -> InvestorProject?`; `changes` is a stream. Favourites use `favouriteIds` and `setFavourite(id, bool) -> Result<void>`; searches use `recentSearches`, `suggestedProjectIds` and `rememberSearch(query)`. `asOf` is the mock catalogue reference date.

Backend work: authoritative project IDs, currency, annual basis-point rate, term/cadence, minimum/capacity/status, filters and stable ordering. Agree cursor pagination before replacing a full-list contract; do not silently drop later pages. Preserve empty results and not-found/forbidden distinctions. Favourites are entity-scoped and need optimistic rollback and last-intent protection. `currentProject` is a local read model, not a network call.

Prove with filter tests [source: test/project_filter_test.dart], search journey [source: test/investor_search_journey_test.dart], home journey [source: test/investor_home_journey_test.dart] and stores tests [source: test/investor_stores_test.dart], including out-of-order favourite responses and scope changes.

Project details, updates and documents

Models: InvestorProject [source: lib/features/investor/domain/investor_project.dart] and mock project content [source: lib/features/investor/data/mock_project_content.dart], loaded by the catalogue interface. `InvestorProjectDetailStore` also resolves investment IDs through the account repository. Documents and updates are model content; there is no production download/upload adapter behind the local preview UI.

Backend work: authorized project/update/document IDs, published versions, availability/status, MIME type/size, safe titles, expiry and signed object access or authenticated streaming. Define gallery content and update pagination; separate unavailable, expired, loading and empty states. Agree download progress/cancellation, malware checks and file retention server-side. Never use a raw client path as object ownership evidence.

Prove with client review fidelity tests [source: test/client_review_fidelity_test.dart], project/document journeys [source: test/client_review_journey_test.dart] and critical journeys [source: test/investor_critical_journey_test.dart]. Test empty/long/unavailable files, changed project during loading and denied/expired access.

Quotes and investment execution

Contracts/models: `InvestorAccountRepository.invest`, `InvestorCatalogRepository.contribute`, InvestmentProjection / InvestmentPolicy [source: lib/features/investor/domain/investment_policy.dart], InvestmentStore [source: lib/features/investor/presentation/stores/investment_store.dart] and InvestorDrafts [source: lib/features/investor/domain/investor_drafts.dart]. The account implementation is MockInvestorAccountRepository [source: lib/features/investor/data/mock_investor_account_repository.dart].

`invest` takes `InvestorProject`, `MonetaryAmount`, `operationId`, `pin`, optional `InvestmentProjection quote` and the demo-only `simulateBiometric` flag, returning `Result<InvestorOperationReceipt>`. `newOperationId()` generates process-local mock IDs. `contribute(id, amount) -> Result<void>` changes local catalogue capacity synchronously. It is not an independent production financial endpoint. A projection carries no server quote ID, expiry or signature today.

Backend work: quote creation/revalidation, quote ID/expiry, exact transport amounts, minimum/maximum and capacity, KYC eligibility, challenge/PIN or provider authorization and atomic investment execution. Allocate robust server-supported idempotency keys and bind keys to the authenticated user/entity and request fingerprint. Concurrent requests and capacity changes after authorization must never overfund a project. Return authoritative transaction/holding IDs and status; keep the UI journey's project/amount/quote together. Retry an uncertain write only using the same backend-supported key and reconcile its result.

Prove with transaction tests [source: test/investor_transaction_test.dart], client correction tests [source: test/client_review_corrections_test.dart], policy tests [source: test/client_review_policy_test.dart] and critical journey [source: test/investor_critical_journey_test.dart]. Exercise fully funded and minimum guards, stale quote, post-PIN capacity change, duplicate/replayed requests, changed key payload, insufficient funds and account switch while pending.

Wallet, deposits, withdrawals and payment methods

Same account contract/implementation. `snapshot()` provides balances, payment methods and account state. `addMoney`/`withdraw` take `MonetaryAmount`, `methodId`, `operationId`, `pin`, optional simulated biometrics and return `Result<InvestorOperationReceipt>`. `addCard` takes card details and `reusable`; `addBankAccount` takes bank/account/type/branch/holder fields and returns `Result<InvestorPaymentMethod>`. Methods also remove/set a default payment method.

Backend work: provider tokenization, approved payment-method setup/verification, pending/failed/settled transitions, fees and available balance. Current raw card fields serve simulated UI validation; agree a compliant provider flow so sensitive card values do not enter general logs or storage. Fee behavior in investor_transfer_fees.dart [source: lib/features/investor/domain/investor_transfer_fees.dart] is a mock assumption (including R1.50 withdrawal fee), not approved server policy. List contracts currently return snapshots; agree scalable pagination if needed. No optimistic notification proves settlement.

Prove with payment-method tests [source: test/investor_payment_methods_test.dart], account tests [source: test/investor_account_repository_test.dart], transaction tests [source: test/investor_transaction_test.dart] and the client journey [source: test/helpers/client_review_journey.dart]. Cover unavailable methods, fee affordability, duplicate writes, pending settlement and status reconciliation.

Transactions, receipts, holdings and portfolio

`transaction(id)`, `investment(id)` and `snapshot()` return the InvestorTransaction / InvestorInvestment / InvestorAccountSnapshot models [source: lib/features/investor/domain/investor_account.dart]. Receipt writes return `InvestorOperationReceipt`; rows carry signed transaction amounts, fees, timestamps, references, status and optional recorded project terms. Current receipt presentation uses local models, not an authoritative signed PDF service.

Backend work: stable IDs, UTC timestamps, cursor pagination/filtering, financial reconciliation, recorded schedules/milestones and whether principal remains held. Confirm positive/negative amount conventions and currency. Home total is available wallet plus distinct held principal; projected payouts must not become additional capital. Monthly simple-interest previews do not rewrite historical quarterly schedules. Separate aggregate proceeds, earned-to-date, principal return and final maturity cash, and avoid duplicate credits or immediate-income promises.

Prove with portfolio analytics [source: test/portfolio_analytics_test.dart], monetary arithmetic [source: test/monetary_amount_test.dart], home journey [source: test/investor_home_journey_test.dart] and transaction tests [source: test/investor_transaction_test.dart]. Add reconciled server examples for partial settlement, reversal, delayed distributions and matured principal.

KYC: Individual, Company, Trust and Stokvel

Contract: KycRepository [source: lib/features/investor/domain/kyc/kyc_repository.dart]. Implementation: MockKycRepository [source: lib/features/investor/data/mock_kyc_repository.dart]. Models/requirements: kyc_models.dart [source: lib/features/investor/domain/kyc/kyc_models.dart], kyc_plan.dart [source: lib/features/investor/domain/kyc/kyc_plan.dart], EntityType [source: lib/features/shared/domain/app_user.dart].

`load(entity) -> Future<Result<KycDraft>>`; `save(draft)` is synchronous. `upload(entity, key, name, bytes, mimeType, cancellation, append, documentId)` and `uploadMemberDocument(...)` return draft results; metadata is simulated and no actual bytes are transported. Delete/save member, allocation and member-draft operations return `Result<KycDraft>`. `invite`, `requestResolutionSignatures`, `submit`, `simulateResolutionSignature`, `simulateRequirementApproval` and `simulateReview` return future draft results. Simulation methods are not acceptable production approvals.

Individual: identity, contact, address, financial/tax and supporting evidence. Company: organisation/representative/member/shareholding information and documents. Trust: trustees, allocations, resolution/signature workflow and supporting evidence. Stokvel: organisation and membership/role/allocation evidence. Use the typed plan's current required field/step keys and saved state; agree provider mapping and compliance rules for all four rather than inferring approval from UI completion counts.

Backend work: durable entity drafts/revisions, upload bytes and type/size validation, secure document IDs, cancellation/retry, provider sessions, authorized invitations/signatures, review/correction status, retention/deletion and authenticated callbacks. Saved changes invalidate related requirement approvals. Local confirmation does not prove identity, review completion or a signature. Extend sync local-edit

contracts deliberately for remote persistence/conflict handling. Member/document lists need agreed limits or pagination.

Prove with KYC tests [source: test/investor_kyc_test.dart], member tests [source: test/investor_member_test.dart], resolution tests [source: test/investor_resolution_test.dart], supporting documents tests [source: test/investor_supporting_documents_test.dart], upload tests [source: test/investor_document_upload_test.dart] and native verification [source: integration_test/investor_verification_test.dart]. Preserve all four entity journeys, allocation guards, replace/delete/multi-document behavior, failed/retried uploads and scope cancellation.

Notifications, profile and security

Investor notification methods are `notification(id)` and `markNotificationsRead([id])` on the account repository. Landowner uses NotificationRepository [source: lib/features/notifications/domain/notification_repository.dart], registered to MockNotificationRepository [source: lib/features/notifications/data/mock_notification_repository.dart]: `list`, `markRead`, `markAllRead`, `messages`, `sendLocalMessage` return `Result` models/lists. Chat is local and sends nothing externally.

ProfileRepository [source: lib/features/profile/domain/profile_repository.dart], registered to MockProfileRepository [source: lib/features/profile/data/mock_profile_repository.dart], supplies `load/update`, `preferences/updatePreferences`, `payoutAccount/updatePayout`, `securityPreferences/saveSecurityPreferences` and `updateSecurity(action, pin)`. Its security/previous-device values are simulated. TransactionAuthorization [source: lib/core/services/transaction_authorization.dart] is a synchronous mock PIN/biometric abstraction, not a secure server authority.

Backend work: unread counts, stable notification/message IDs, cursor pagination, preferences and channel/topic opt-out, authorized profile updates and audited security/device operations. Agree asynchronous real MFA/PIN/device revocation flows; never send a `simulateBiometric` or `simulateReview` bypass as authority. See notifications and installation registration.

Prove with security-state tests [source: test/security_state_test.dart], profile-security tests [source: test/investor_profile_security_test.dart], notification tests [source: test/notifications_fidelity_test.dart] and profile tests [source: test/profile_fidelity_test.dart]. Test logout cleanup, masking semantics, unread state and inaccessible notification targets.

Landowner properties, submissions, documents, earnings and payouts

Contract: LandownerRepository [source: lib/features/landowner/domain/landowner_repository.dart]. Implementation: MockLandownerRepository [source: lib/features/landowner/data/mock_landowner_repository.dart]. `overview -> Result<LandownerOverview>`, `properties -> Result<List<Property>>`, `property(id) -> Result<Property>`, `earnings -> Result<EarningsSummary>`. `readDraft(id)` and `saveDraft(SubmissionDraft)` are synchronous local operations; `submit(draft) -> Result<String>` returns the created local property ID. `changes` and balance visibility streams synchronize independent stores. Models: property.dart [source: lib/features/landowner/domain/property.dart], SubmissionDraft / LocalAttachment [source: lib/features/landowner/domain/submission_draft.dart].

Backend work: authorized property ownership, stable draft/property/document IDs, resumable/versioned drafts, actual file transfer, review/offer/status transitions, pagination and server validation. `LocalAttachment` carries sample metadata; no upload bytes or document service is implemented. Financial form strings require validated precise decimal transport and approved server conversion. Earnings summaries are fixture read models, not settlement. Payout account editing uses shared `ProfileRepository.updatePayout(PayoutAccount)`; add server verification, payout execution/status and reconciliation contracts explicitly.

Prove with submission tests [source: test/submission_fidelity_test.dart], stores tests [source: test/stores_test.dart], payout tests [source: test/payout_fidelity_test.dart], dashboard tests [source: test/dashboard_fidelity_test.dart] and native Landowner journey [source: integration_test/landowner_flow_test.dart]. Test resume/back/portal switching, repeated submit, missing documents, failed writes and expired sessions without losing or misattributing server

drafts.

Small adapter example (proposal)

This is a **method-level illustration** matching the real `InvestorCatalogRepository.projects()` return type. It is not a complete implementation of that interface. The endpoint path and decoder must be supplied from agreed backend contracts; no endpoint is invented or added to the app.

```
final class CatalogueReadAdapter {
  CatalogueReadAdapter(this.client, this.agreedPath, this.decodeProjects);

  final ApiClient client;
  final String agreedPath;
  final List<InvestorProject> Function(Object?) decodeProjects;

  Future<Result<List<InvestorProject>>> projects() {
    return client.request<List<InvestorProject>>(
      path: agreedPath,
      decode: decodeProjects,
    );
  }
}
```

The complete adapter must also implement scope cancellation, current read models, streams, favourites/search behavior and safe disposal, and make pagination explicit. For these Dart types, see ApiClient [source: lib/core/network/api_client.dart] and catalogue contract [source: lib/features/investor/domain/investor_repository.dart]. Keep store constructors unchanged where the existing contract suffices.

Transport, authorization and settlement rules

`ApiClient` requires an HTTPS base and a local absolute request path; it disallows request authorities/schemes and redirects. It uses 15/20/30-second connect/send/receive timeouts, optional bearer attachment, `CancelToken`, typed error mapping [source: lib/core/network/api_error_mapper.dart] and safe method-only debug logging. GET/HEAD can retry once after 200 ms for selected connection failures or 502/503/504; other methods receive one attempt. Keep these restrictions. Single-flight token refresh, refresh rotation, coordinated expiry/revocation cleanup and server error-field decoding are still to be implemented. Token read failure is unauthenticated; the server must reject missing/invalid credentials.

Agree integer cents versus strictly validated decimal-string transport, currency code, basis points and rounding before decoding money. Do not introduce binary floating-point ledger arithmetic. The server owns balances, quotes/rates/fees, capacity, KYC eligibility, settlement and final authorization. UI disabling protects a journey, not an account.

Provider webhooks require authenticated signature verification, replay protection, idempotency and reconciliation on the backend. Handle pending/failed/cancelled/settled states explicitly. Never infer settlement from push payloads, optimistic screens or a local authorization result. Reconcile uncertain writes and reserve capacity atomically with execution.

Accepted behavior and production decisions

The accepted UI remains intact: Rand/ZAR, R1 minimum, monthly new-project projections, 36-month policy, literal masks, Total Earnings to date and the compact calculator banner. These are not reopened as unresolved visual decisions.

Production input is still needed for canonical catalogue/gallery content and rates, the final withdrawal fee policy, legally approved copy/timing, Trust signature/provider rules and content selection for Home promotions. Runtime currently shows one calculator promotion; an alternate portfolio variant remains available in source. Runtime Trust counts reflect saved valid records rather than contradictory sample

counts. Aggregate proceeds and maturity labels need a consistent server/read-model vocabulary. These inputs should refine live data/contracts while preserving accepted presentation.

Recommended implementation order

1. Confirm environments, contracts, authentication and entity authorization.
2. Connect authentication/profile and one read-only catalogue journey.
3. Connect wallet/account/portfolio read models and reconciliation.
4. Connect KYC and Landowner drafts/documents/submissions.
5. Connect server quotes, transaction authorization and financial writes.
6. Connect installation registration, notifications, monitoring and release controls.
7. Run integration, security and failure-path tests in staging before production.

For each replacement, rerun the full maintained suite and add contract tests against agreed staging responses. Prove authorization and concurrent/idempotent financial behavior server-side; passing Flutter tests alone cannot establish those properties.

Observability and notifications

Documentation entry point

This is a recommended next phase, not an enabled collection or delivery pipeline. No cloud resources, permissions, identifiers or new packages were activated for handover. Official documentation was consulted on 22 September 2026; the API names below were also checked against the delivered packages, rather than assumed from newer examples.

Existing service adapters

telemetry.dart [source: lib/core/services/telemetry.dart] defines `AnalyticsService`, `CrashReporter`, `FeatureConfigService` and `PerformanceService`, with Firebase and no-op implementations. Locked versions: firebase_core 4.14.0, firebase_analytics 12.5.0, firebase_crashlytics 5.3.0, firebase_performance 0.11.5 and firebase_remote_config 6.6.0. push_notification_service.dart [source: lib/core/services/push_notification_service.dart] wraps onesignal_flutter 5.6.8.

Firebase is disabled by default. `TelemetryServices.initialize` accepts optional `FirebaseOptions`, initializes once, configures collection gates and returns no-ops if setup fails. The composition root currently supplies no platform options. Add approved environment configuration through the existing initialization path; avoid another independent initializer. The analytics wrapper supports `screen(name)` but no app screen-event call sites are wired. Performance supports `trace(name, operation)` but workflows are not instrumented. Crash hooks are connected to the disabled reporter. Remote Config has a typed default and fetch adapter; it is not a business authorization service. Official Flutter setup explains platform registration/options and native product configuration.

OneSignal initializes only with a nonempty app ID, registers click/foreground listeners, exposes permission/login/logout/tag methods, and removes listeners on disposal. No app ID, device-registry API or native production transport setup is delivered. It can initialize independently of Firebase/mock flags: leave `ONESIGNAL_APP_ID` explicitly empty during demo verification.

Analytics proposal

Extend `AnalyticsService` with a small typed event interface using the pinned SDK's `logEvent`; retain `logScreenView` for screen names. Track the effective visible named route centrally around go_router, including changes between nested portal tabs. Deduplicate consecutive identical screen names and avoid counting both a shell and its child as separate screen impressions. Use stable labels such as `investor_project`, never full paths or IDs. The screen observer/event layer is future work, not implemented tracking. See Analytics setup and screen-view guidance.

Proposed event dictionary (product owner approves definitions; mobile team emits once at the stated trigger):

Event	Trigger	Allowed parameters
`onboarding_step`	A user reaches a new onboarding step	`portal`, bounded `step`
`sign_in_outcome`	An attempt completes	`method`, `outcome` enum; no credential/error text
`portal_switch`	User completes an app-switcher selection	`from_portal`, `to_portal`
`project_view`	A project screen resolves successfully	`category` enum; no project/account IDs

Event	Trigger	Allowed parameters
`calculator_use`	User completes a valid preview	`entry_point`; no principal, rate or result values
`submission_progress`	A persisted step is acknowledged	`workflow`, bounded `step`, `outcome`
`notification_open`	One validated tap is accepted for handling	`destination` enum, `app_state` enum

Do not log balances, exact investments, email/phone, PIN/OTP, tokens, bank/card data, tax information, KYC answers or document contents. Avoid free text and high-cardinality IDs. Treat analytics as usage measurement, not the financial ledger or compliance audit log; backend operational auditing needs a separate controlled design.

The existing build gate is not a consent screen. Implement approved consent and runtime collection controls using the pinned `setConsent` and `setAnalyticsCollectionEnabled` APIs, including native defaults where necessary. Use distinct debug/staging data and verify events in DebugView, disabling debug mode after checking. On logout/account change, clear analytics identity (`setUserId(id: null)`) and any user properties added by the implementation. The current logout path does not do this because analytics identity is not currently set. Product/privacy owners approve collection; mobile and analytics owners verify event counts and exclusions before rollout.

Crashlytics proposal

bootstrap [source: lib/app/bootstrap.dart] already installs `FlutterError.onError` and `PlatformDispatcher.instance.onError`. Extend that single path rather than adding duplicate global reporters. It reports framework/platform errors as fatal today. `FirebaseCrashReporter.record` sends error runtime type and the original stack, avoiding raw exception values. Improve redaction while preserving enough sanitized context to diagnose unexpected failures; do not record routine validation/cancellation as crashes. Background isolates, native crashes and handled exceptions need appropriate coverage and verification, not an assumption that every error is automatically captured. See Crashlytics Flutter setup and custom reporting.

Use safe breadcrumbs and low-cardinality release/version/build/environment tags. Exclude user-entered text, paths with identifiers, payloads and credentials. Report handled unexpected failures deliberately with stacks and nonfatal classification. Review the existing all-fatal global handling as part of integration, without duplicating a report in the repository and global hook.

Complete Android product plugin/mapping integration and iOS Crashlytics symbol upload after approved Firebase registration. Current Android release builds use the existing debug signing and no Dart obfuscation flags. If future builds use `--obfuscate --split-debug-info`, retain exact Dart symbols together with release metadata; preserve native mapping files/dSYMs matching each release. Confirm symbolication on both platforms. Flutter obfuscation guidance explains the Dart symbol requirement.

In a staging-only build, trigger one controlled test exception, relaunch if required, and verify a single correctly symbolicated report with sanitized context and the right build/environment. Remove the trigger from normal distributed builds. Assign an on-call owner, alert routing, regression threshold and release-triage responsibility before enabling collection.

Performance proposal

`FirebasePerformanceService.trace` already stops its trace in `finally`, including failed operations. Add bounded traces for startup and important catalogue/account/submission loads. Measure the actual Dio transport [source: lib/core/network/api_client.dart]: the pinned SDK supports `FirebasePerformance.newHttpMetric`. Instrument one transport boundary if requests are not already measured; do not add overlapping interceptor and repository metrics for the same request. Close each metric on success, error and cancellation, with response code and safe payload sizes where available. Redact dynamic URL paths/query values and use approved endpoint templates. Performance setup and custom network monitoring cover native setup and manual metrics.

Keep names/attributes bounded (`environment`, route category, outcome); never attach balances, user/entity IDs, document names or tokens. Separate initial startup from background work and use staging samples to validate counts before rollout.

Firebase Performance does not automatically measure each Flutter screen as a distinct native rendering screen. Use profile-mode Flutter DevTools frame analysis on representative physical devices for rendering/jank, with the accepted banner and large-text layouts included. Simulator/debug timings are not production performance evidence. Flutter performance profiling describes this workflow.

Proposed installation registry

No backend installation registry is implemented. Add an authenticated API after agreeing schema and lifecycle:

- An **installation ID** is an app-scoped opaque random identifier for an installation, not IMEI, advertising ID, fingerprint or proof of physical-device identity. Persistence across reinstall varies by platform/storage and must not be assumed.
- A **backend device-record ID** identifies the registry row. A **session ID** identifies one authenticated session. **User/entity IDs** identify authorized ownership/scope. A **push subscription ID/token** identifies a provider transport and can rotate independently.
- Upsert platform, app version/build, bounded supported metadata, permission state, provider/subscription identity and timestamps. Permit registration before permission or token availability. Derive ownership from authentication, not an arbitrary client-supplied user ID.
- Handle subscription rotation, multiple devices, account changes, logout unbinding, revocation and stale-record removal. Retain only agreed metadata and make retries idempotent. The server validates updates to records already bound to another account.

`OneSignal.User.pushSubscription.addObserver` exists in 5.6.8 and can drive subscription updates; the current abstraction does not expose it. Extend the abstraction and dispose observers correctly. Do not label the mock profile's previous-device list as this registry.

Push delivery and navigation

Prefer completing the existing OneSignal adapter unless the CTO explicitly selects direct FCM. OneSignal still needs Android FCM and iOS APNs configuration, approved App ID, platform registration, capabilities and any required notification service extension/app-group setup. No native service extension or production APNs entitlement is delivered. Follow OneSignal Flutter setup and the mobile SDK reference.

Ask for notification permission at a contextual user action, with consent and useful denial/settings behavior. The existing wrapper calls `requestPermission(false)` and safely returns false when unconfigured. Bind identity after restored/new login and update/unbind on account/logout transitions. `DevelopifyApp` currently listens for later session changes but does not explicitly synchronize the already-restored identity on first attachment; address that when completing push. The pinned SDK provides

`OneSignal.loginWithJWT(externalId, jwt)` for a backend-issued identity-verification token; the current wrapper uses `login`. Agree provider-side identity verification and server issuance before exposing verified login through the abstraction. Signing secrets remain backend-only.

Current temporary typed payloads are `destination: notifications` or `destination: property` with an ID matching 1-80 letters/digits/underscore/hyphen. `PushTarget.fromPlaceholder` rejects other shapes. Current tap handling navigates directly to Landowner routes; router guards can defer a protected location during login. This is not a durable cold-start queue, entity authorization check, arbitrary deep-link parser or Investor notification route contract. Foreground targets are exposed as a stream but no app subscriber/presentation policy is wired.

Proposed completion: agree versioned typed payloads and stable event IDs; deduplicate repeated taps/deliveries; handle foreground/background/terminated startup; defer until router, session and authorized entity scope are ready. Resolve only primitive project/document IDs after authenticated resource checks. Reject stale/inaccessible targets, handle permission denial and sync unread counts/preferences from the backend. Keep notification bodies non-sensitive. Refresh authoritative state after a tap; never trust pushed amounts or settlement status.

An FCM-only choice requires replacing OneSignal identity/subscription/listener handling, adding and configuring firebase_messaging, platform/background handlers and a single presentation/navigation pipeline. Do not run competing OneSignal and FCM presentation pipelines by default. See FCM Flutter setup.

OneSignal REST keys, APNs signing keys, service-account credentials and server secrets never belong in Flutter source or the source ZIP.

Remote Config and tester distribution

`FirebaseFeatureConfig` seeds `review_business_days=10`, validates fetched values to 1-30, uses a 10-second fetch timeout and 12-hour minimum interval, and falls back safely. Proposed use: non-sensitive feature availability, maintenance/version messages and rollout controls with typed defaults. It must not grant authorization, override server financial/KYC limits or contain secrets. The adapter's `refresh()` calls `fetchAndActivate`; no release policy or UI fetch lifecycle is wired. Remote Config Flutter guide provides setup details.

Firebase App Distribution is a future tester/CI workflow, not a runtime SDK requirement. Prepare approved app registrations, signing, tester groups, least-privilege CI credentials, checksums/release notes, matching symbols and staging gates; then verify delivery and tester install before calling a pipeline operational. Production signing and store accounts remain CTO-owned. See App Distribution.

Recommend separate development, staging and production Firebase projects, while retaining both portals in the same app. The CTO must approve environment strategy, retention, consent and collection policy before activation. Firebase environment best practices supports this separation.

Handover status

Documentation entry point

Delivered scope and identity

Developify Mobile **1.0.0+6**, handover revision **r1**, prepared 22 September 2026 for Ngoako Masetla. One app contains the accepted Investor and Landowner UI, shared authentication and portal switching. The accepted compact calculator-banner correction is included from baseline `0f417ec2fa7ff3fa4cf801f190d757a2052c345e` on `main`. The application version was not changed for documentation.

The source ZIP is exported from the final cleanup commit. Its exact revision is recorded in .source-revision; Git archive expands that file's revision marker. The accompanying checksum file identifies the same source commit and final ZIP/PDF hashes. The source package contains no repository history.

Runtime assets, SVG identifiers, image crops, TASA font licence, native launcher/splash resources, repository architecture and generated MobX source are preserved. No runtime asset or dependency/lockfile was changed. All maintained test assertions remain. Internal review material and editor state were removed from the working project. Optional maintained-test screenshot support now writes ignored build output and no longer exports internal geometry/asset inventories.

Current service and feature state

Area	Delivered state
Investor/Landowner screens, validation, routing, privacy, accessibility	Implemented UI/domain behavior; accepted presentation retained
Authentication/recovery/social sign-in	Simulated; mock login can restore securely for its unexpired demo session
Wallet, investment, deposits, withdrawals, earnings and receipts	Simulated data/operations; no production money movement
KYC, invitations, signatures and document previews	Simulated; no live identity, signature or document service
Repository interfaces, Dio/token storage	Existing integration skeleton; live repositories and token refresh still need implementation
Firebase analytics/crash/performance/config	Existing optional adapters; configuration required; collection disabled
OneSignal	Existing optional adapter; App ID empty; platform/provider integration required
Installation registry, analytics event dictionary, CI tester distribution	Recommended future implementation
Android signing	Existing debug signing, including release-mode local QA; no production signing delivery
iOS signing	Simulator verification only; production team/provisioning remains CTO-owned

Verification uses `MOCK_MODE=true`, `MOCK_SCENARIO=success`, `FIREBASE_ENABLED=false`, `TELEMETRY_ENABLED=false`, and explicit empty `ONESIGNAL_APP_ID`. No production API, push or collection is activated. Setting `MOCK_MODE=false` or adding an API URL is not sufficient to connect services: registered mock requests become unavailable.

Verification record

The accepted baseline was freshly regenerated, format-checked and analyzed with Flutter 3.44.4 / Dart 3.12.2; all 754 maintained tests passed. The final cleaned-source verification and native results are recorded below before source export. The accompanying checksum file records the fresh-extraction results for the exact finished archive, separately from the baseline. Follow the commands in Setup to reproduce them.

Check	Fresh result
Locked Pub dependencies	Passed `flutter pub get --enforce-lockfile`; app, Pub and Pod lockfiles unchanged
Generation / formatting / analyzer	Passed; generation wrote zero outputs; no analyzer issues
Full maintained host suite	754 tests passed after cleanup

Check	Fresh result
Runtime and assertion audit	279 Dart files have identical non-comment tokens to baseline; all 2,380 assertion calls unchanged
Focused banner checks	All 12 width/text-scale cases passed; two saved before/after banner PNGs are byte-identical
Normal mock Android build	Debug APK built successfully with optional services explicitly disabled
Native Android review journey	Both tests passed: money/privacy/project guards/documents and portal switching
Native Android Landowner journey	Both tests passed, including login/recovery, draft resume/submission and switching
iOS simulator	Build blocked under Xcode 27: universal framework architecture check failed (exit 255); selected-simulator attempt reported dependency deployment targets below iOS 15. No fresh iOS journey success claimed
Source/package checks	Relative documentation links checked; private-path/credential/residue audit and `git diff --check` passed before export

The Android build reports a future Kotlin Gradle Plugin compatibility warning for Firebase plugins. The pinned toolchain builds successfully; no upgrade or native migration was performed. The Xcode limitation and reproduction command are in the setup guide. Source builds are temporary validation artifacts and are excluded from delivery.

These checks verify the delivered mock application and source reproducibility within the recorded toolchain. They do not prove production API authorization, financial settlement, live KYC or store readiness. Historical delivery results are not used as fresh native verification.

First integration steps and access inputs

1. Confirm environment/API contracts, identity provider, user/entity authorization and session expiry/revocation cleanup.
2. Connect authentication/profile and one read-only catalogue journey, retaining repository interfaces and independent stores.
3. Reconcile wallet/account/portfolio reads before enabling financial writes; agree quotes, idempotency, fees, capacity and final server authorization.
4. Supply durable draft/document/KYC contracts for all four entity types and Landowner submissions.
5. Confirm Firebase/OneSignal projects, privacy/consent policy, installation registry, notification payloads and monitoring owners.
6. Supply signing/store/CI access and prove staging failure paths before production release.

The API guide lists exact interfaces, methods, tests and backend inputs. The service guide provides recommendations and current official links. These production inputs do not block delivery of the accepted runnable mock source.

Delivery consists of a source ZIP with Markdown documentation, a standalone developer PDF and a checksum file. File sharing follows the owner's instruction: **Anyone with the link, Viewer/download access** for the new files; the existing parent folder permissions and previous deliveries remain unchanged. No automatic email or group message is sent. No source push, live service activation or store deployment is part of this handover.